High-Level Design (HLD)

RAG-Based Customer Support Assistant using LangGraph and HITL

1. System Overview

The system is designed as an intelligent customer support assistant that can understand and answer user queries based on uploaded documents. Instead of relying on predefined responses, it uses a Retrieval-Augmented Generation (RAG) approach to provide accurate and context-aware answers.

When a user uploads documents such as PDFs or text files, the system processes them, converts the content into embeddings, and stores them in a vector database. When a question is asked, the system retrieves the most relevant information and uses a Large Language Model (LLM) to generate a meaningful response.

To make the system more reliable and production-ready, a workflow layer using LangGraph is introduced. This layer helps in controlling the flow of execution and deciding whether a response should be returned directly or escalated to a human (HITL) when confidence is low.

2. Problem Definition

Traditional customer support systems often fail to provide accurate answers when dealing with large or dynamic knowledge bases. They either depend on static FAQs or generate generic responses that lack context.

This project aims to solve that problem by:

- Providing document-based, context-aware answers
- Reducing hallucination in AI responses
- Introducing a fallback mechanism (HITL) when the system is unsure

3. Scope of the System

The system focuses on building a smart assistant capable of handling document-based queries.

Included in Scope:

- Upload and processing of PDF/TXT documents
- Semantic search using embeddings
- Context-aware response generation
- Workflow-based execution using LangGraph
- Human-in-the-loop escalation

Not Included:

- Full enterprise authentication systems

- Integration with external customer support tools
- Real-time human chat system

4. System Architecture

The system follows a modular architecture where each component is responsible for a specific task.

User → Upload Documents → Process Data → Store Embeddings
→ Query → Retrieve Context → Generate Answer
→ Decision Layer → (Answer OR Human Escalation)

The workflow ensures that the system is not just answering questions but also making decisions based on confidence and context availability.

5. Component Description

User Interface

The front-end is built using Streamlit, providing a simple chat-based interface. Users can upload documents and ask questions in a conversational format.

Document Processing

Uploaded files are parsed and converted into text. This forms the raw knowledge base for the system.

Chunking Strategy

The text is divided into smaller chunks to improve retrieval accuracy. Overlapping chunks are used to ensure that important context is not lost.

Embedding Model

Each chunk is converted into a numerical representation using a sentence transformer model. This allows semantic comparison between queries and stored data.

Vector Database (FAISS)

The embeddings are stored in FAISS, which enables fast similarity-based search.

Retrieval Layer

When a query is received, the system retrieves the most relevant chunks from the vector database.

LLM (Groq)

The retrieved context is passed to the LLM, which generates a natural and meaningful response.

Workflow Engine (LangGraph)

LangGraph acts as the control system of the application. It defines how data flows between different components and enables conditional decision-making.

Decision Layer

This layer evaluates the response confidence. Based on this, it decides whether to return the answer or escalate.

HITL (Human-in-the-Loop)

If the system is not confident or lacks sufficient context, the query is escalated to a human. This ensures reliability and trustworthiness.

6. Data Flow

The data flow in the system follows a clear sequence:

1. The user uploads a document
2. The document is processed and split into chunks
3. Each chunk is converted into embeddings
4. Embeddings are stored in the vector database
5. The user asks a question
6. Relevant chunks are retrieved
7. The LLM generates an answer using the context
8. The system evaluates confidence
9. If confidence is low, the query is escalated

7. Technology Choices

The choice of technologies is based on performance, flexibility, and ease of integration.

- **FAISS** is used for fast and efficient vector search
- **LangGraph** is used for managing workflow and decision logic
- **Groq LLM** is used for fast and high-quality response generation

- **Sentence Transformers** are used for generating embeddings

These tools together enable the system to be both scalable and efficient.

8. Scalability Considerations

To ensure the system can handle growth, several factors are considered:

- **Large Documents:** Chunking ensures that large files can be processed efficiently
- **High Query Load:** The vector database can scale to handle more queries
- **Latency:** Efficient retrieval and fast LLM inference help reduce response time

Future improvements can include caching, distributed databases, and API-based scaling.

Final Note

This system is not just a chatbot. It is a structured AI system that combines retrieval, generation, workflow control, and human fallback to deliver reliable and context-aware responses.

Low-Level Design (LLD)

RAG-Based Customer Support Assistant using LangGraph and HITL

1. Overview

The Low-Level Design focuses on how each component of the system is implemented and how they interact internally. While the High-Level Design explains *what the system does*, the LLD explains *how it does it*.

The system is divided into modular components such as document processing, embedding generation, retrieval, response generation, and workflow execution using LangGraph.

2. Module-Level Design

The system is structured into multiple modules, each handling a specific responsibility.

2.1 Document Processing Module

This module is responsible for handling user-uploaded files.

- Accepts PDF and TXT files
- Converts them into readable text format

- Ensures encoding compatibility

👉 Output: List of raw documents

2.2 Chunking Module

After loading the document, the text is divided into smaller chunks.

- Uses fixed chunk size (e.g., 800 characters)
- Includes overlap (e.g., 100 characters)
- Prevents loss of context between chunks

👉 Output: List of text chunks

2.3 Embedding Module

Each chunk is converted into a vector representation.

- Uses sentence-transformer model
- Converts semantic meaning into numerical vectors

👉 Output: Embedding vectors

2.4 Vector Storage Module

Stores embeddings efficiently for fast retrieval.

- Uses FAISS as vector database
- Supports similarity search

👉 Output: Indexed vector store

2.5 Retrieval Module

Handles query-based search.

- Takes user query
- Converts query into embedding
- Retrieves top-k relevant chunks

👉 Output: Relevant context

2.6 Query Processing Module

Prepares input for the LLM.

- Combines retrieved context with user query
- Formats prompt
- Ensures clarity and constraints

👉 Output: Structured prompt

2.7 LLM Module

Generates responses using Groq.

- Receives prompt
- Produces natural language answer
- May include confidence estimation

👉 Output: Answer text

2.8 Graph Execution Module (LangGraph)

Controls the workflow of the system.

- Defines nodes (retrieve, generate, decision, human)
- Manages execution flow
- Enables conditional routing

👉 Output: Final system response

2.9 HITL Module

Handles escalation when the system is unsure.

- Triggered when confidence is low
- Returns escalation message
- Can be extended to real human support

👉 Output: Escalated response

3. Data Structures

3.1 Document Representation

```
{
  page_content: string,
```

```
  metadata: {source: file_name}
}
```

3.2 Chunk Structure

```
{
  chunk_text: string,
  chunk_id: int
}
```

3.3 Embedding Structure

```
{
  vector: [float],
  dimension: 384
}
```

3.4 Query-Response Format

```
{
  query: string,
  context: string,
  answer: string,
  confidence: float
}
```

3.5 Graph State Object

```
{
  question: string,
  context: string,
  answer: string,
  confidence: float
}
```

4. Workflow Design (LangGraph)

The system workflow is implemented using LangGraph.

Nodes:

- Retrieve Node
- Generate Node
- Decision Node

- Human Node

Flow:

Input → Retrieve → Generate → Decision → Output / Human

State Flow:

- Question enters system
- Context is added
- Answer is generated
- Confidence is evaluated

5. Conditional Routing Logic

The system decides the output path based on confidence and context.

Conditions:

- If relevant context is found → Generate answer
- If confidence is high → Return answer
- If confidence is low → Escalate

Example:

IF confidence > 0.6 → Return Answer
ELSE → Trigger HITL

6. HITL Design

The Human-in-the-Loop mechanism ensures system reliability.

When triggered:

- Low confidence
- No relevant data
- Complex or ambiguous query

What happens:

- System stops automated response

- Returns escalation message
- Can be extended to human agent interface

Integration:

- HITL is part of LangGraph workflow
- Acts as final fallback node

7. API / Interface Design

Even though the current system is UI-based, it can be extended into an API.

Input:

```
{
  "query": "What is refund policy?"
}
```

Output:

```
{
  "answer": "...",
  "confidence": 0.8
}
```

Interaction Flow:

- User → UI
- UI → Backend logic
- Backend → RAG + Graph
- Graph → Response

8. Error Handling

The system includes basic error handling to improve robustness.

Missing Data

- If no document uploaded → prompt user

No Relevant Context

- Return fallback response
- Avoid hallucination

LLM Failure

- Catch exception
- Return safe message

File Errors

- Handle encoding issues
- Validate file type

Final Note

The LLD ensures that the system is modular, maintainable, and scalable. Each component is clearly defined, making it easier to debug, extend, and optimize.